

The Admission Monoid:

An Algebra of Refusal

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Pillar I, Paper 01

Abstract

The foundations paper of this series established that admission cannot be a semantic decision and must be a computable retraction, with *refusal* promoted from an exception to a first-class value [?]. This paper develops the algebra of that value. We take as our object the *denial word* — the machine-checkable record of *why* an observation was not admitted — and prove that the space of denial words is a bounded, commutative, idempotent monoid (a join-semilattice) whose identity is the clean word **ADMITTED** and whose product is componentwise disjunction, realized in **praxis** as the **DenialPolarity** bitfield under **compose**. We show that obligations are *lane maps* into this monoid, that total denial is their join, and that admission is *antitone* in the obligation set: adding obligations can only shrink the admitted set (monotonicity). We then formalize the eight-bucket **RefusalCategory** taxonomy as a total classification $\text{cat} : \mathcal{S} \rightarrow \mathcal{C}$ and prove a *mechanized totality theorem*: **cat** is realized by a wildcard-free **match**, so it is compiler-certified total, its image is exactly the seven inhabited buckets, and the eighth (**Reserved**) carries an empty preimage by design — an honest slack coordinate, not an overclaim. We identify precisely where the type system’s totality guarantee ends and a runtime test must continue: the lane decoder **scenario_for_denial_lane** is a *partial* map because its domain is an open 2^{64} newtype rather than a closed enum. We prove that admission composes across a manufacturing pipeline as the unique monoid homomorphism from the free monoid on stages into the denial monoid, and that this composition is order- and multiplicity-independent, so a pipeline admits exactly when every stage admits. Finally, we refine admission at the logical layer: the **prolog8** kernel decides bounded Horn queries by SLD-resolution and returns a *proof-carrying trichotomy* — an **Answered** query ships a positive proof DAG, a **Denied** query ships a negative proof tree, and an **Invalid** query ships a **RejectionCode**; we prove this trichotomy and show it embeds back into the denial monoid. Every object is anchored to a line of the **praxis** repository, so a reader can trace each theorem to the code that enforces it.

Contents

Chapter 1

Introduction: The Algebra a Refusal Needs

1.1 From “refusal is data” to an algebra of refusal

The prior published work [?] established the governing identity $\mathcal{A} = \mu(\mathcal{O}^*)$ and demonstrated it at enterprise scale. The foundations paper of this series (Paper 00) grounded it: it proved a specialization of Rice’s theorem [?] to observations, concluded that admission *cannot* decide meaning and *must* retract onto a decidable sub-language, and promoted the *refusal* $\perp$ from a thrown exception to a returned value. The keystone thesis introduced, in a single Construction, the *denial monoid* $D = (\{0, 1\}^n, \vee, \mathbf{0})$ and one proposition about it. This paper pays that Construction its full debt.

The reason a refusal needs an algebra, rather than merely a name, is scale. Among a fleet of $\sim 10^{12}$ agents, each with bounded context κ and none able to read another’s interior, a refusal must satisfy four demands simultaneously:

- (D1) **Composition.** A manufacture passes through many admission stages; each may refuse; the fleet needs *one* verdict that faithfully aggregates all of them.
- (D2) **Classification.** A refusal must land in a *finite, shared, legible* vocabulary, so that a recipient who never saw the sender’s interior can nonetheless act on the *kind* of refusal.
- (D3) **Totality.** No refusal may fall through a crack: every failure mode must map to a class, and the mapping must be checkable by machine, not by hope.
- (D4) **Reason-carrying.** A refusal must carry *why*; at the logical layer this sharpens to carrying a *proof*.

Composition (D1) wants a monoid. Classification (D2) wants a surjection onto a small set. Totality (D3) wants a mechanized exhaustiveness argument. Reason-carrying (D4) wants, at its limit, proof-carrying evaluation. This paper supplies each in turn — Chapters ?? and ?? for composition, Chapter ?? for classification and totality, and Chapter ?? for proof-carrying refusal — and anchors each to running code.

1.2 First principle

Axiom 1.1 (Refusal is data). There is a distinguished refusal value $\perp$ and a space of *reasons* $\mathcal{R}_f$. A refusal is not the absence of an output but the pair $(\perp, r)$ with $r \in \mathcal{R}_f$ a machine-checkable reason. Admission is therefore *total* as a map into $\mathcal{O}^* \cup (\{\perp\} \times \mathcal{R}_f)$: a correctly functioning admission always returns, either an admitted observation or a refusal-with-reason.

Axiom ?? is the design decision made concrete in **praxis** by the **Andon** type, whose **Halted** variant carries both the unmet obligations and their refusal-taxonomy classification:

```
Andon::Halted{ unmet: Vec<Obligation>, refusals: Vec<RefusalScenario>, at: u64 }.
```

The remainder of this paper is the study of the structure that $\mathcal{R}_f$ carries. We fix, once, the code objects under study: the **DenialPolarity** word and its **compose** operation (**crates/praxis-core**, re-exported from **bcinr-powl-receipt**); the **Obligation** enum and the **Judge/Admit** traits (**law.rs**); the **RefusalScenario/RefusalCategory** taxonomy and its total maps (**refusal.rs**); and the **prolog8 Kernel::query** logic gate (**ops.rs**). Each theorem below names the artifact it governs.

Chapter 2

The Denial Semilattice

2.1 The abstract denial word

Definition 2.1 (Denial word space). Fix $n \in \mathbb{N}$ lanes. The *abstract denial word space* is $\mathbb{D}_n = \{0, 1\}^n$ with componentwise disjunction $\vee$ and the all-zero word $\mathbf{0}$. A lane i is *set* in d when $d_i = 1$; the *support* $\text{supp } d = \{i : d_i = 1\}$ names the fired lanes. Write $d \preceq d'$ for the componentwise order $\forall i (d_i \leq d'_i)$.

Proposition 2.1 ($\mathbb{D}_n$ is a bounded idempotent commutative monoid). $(\mathbb{D}_n, \vee, \mathbf{0})$ is a commutative monoid in which every element is idempotent ($d \vee d = d$). It is the join-semilattice of the Boolean lattice $(2^{[n]}, \subseteq)$ under the support isomorphism $d \mapsto \text{supp } d$; $\mathbf{0}$ is its least element (bottom), and the all-ones word $\mathbf{1}$ its greatest. Consequently $d \vee d'$ is the least upper bound of d, d' , and $\preceq$ is exactly the algebraic order $d \preceq d' \iff d \vee d' = d'$.

Proof. Disjunction is associative, commutative, and idempotent per coordinate, with unit 0 per coordinate; a finite product of such structures inherits all four laws, giving a bounded commutative idempotent monoid. The map $d \mapsto \text{supp } d$ carries $\vee$ to $\cup$ and $\mathbf{0}$ to $\emptyset$, and is a bijection $\mathbb{D}_n \rightarrow 2^{[n]}$, hence a monoid isomorphism onto $(2^{[n]}, \cup, \emptyset)$. In any join-semilattice the identity $d \preceq d' \iff d \vee d' = d'$ holds by antisymmetry of $\subseteq$; least/greatest elements are $\emptyset$ and $[n]$, i.e. $\mathbf{0}$ and $\mathbf{1}$ [?, Ch. I]. $\square$

Proposition ?? is the algebra behind “a refusal category is a join-support” in the keystone: $\text{supp } d(o) \subseteq [n]$ is a subset, and subsets join by union. We now show that the **praxis** implementation realizes exactly this structure.

2.2 The concrete word: DenialPolarity

Definition 2.2 (The polarity word). In **praxis** the denial word is **DenialPolarity**, a `#[repr(transparent)]` newtype over `u64` carved into eight byte-lanes. The clean word is `ADMITTED = DenialPolarity(0)`. Seven named nonzero constants each occupy one distinct byte lane:

<code>PRECONDITION_FAILED</code>	(lane 1)	<code>RESOURCE_EXHAUSTED</code>	(lane 4)
<code>SLA_BREACH</code>	(lane 2)	<code>OBJECT_LIFECYCLE_VIOLATION</code>	(lane 5)
<code>AUTHORIZATION_DENIED</code>	(lane 3)	<code>CONFORMANCE_GATE_FAILED</code>	(lane 6)
		<code>WATCHDOG_DRAINED</code>	(lane 7).

The product is $\text{compose}(a, b) = \text{DenialPolarity}(a.0 \mid b.0)$, bitwise OR; the admission predicate is $\text{is_admitted}(d) \iff d.0 = 0$.

Proposition 2.2 (The polarity word is the denial semilattice). *Let $\mathbb{D} = (\{\text{DenialPolarity}\}, \text{compose}, \text{ADMITTED})$. Then $\mathbb{D}$ is a bounded commutative idempotent monoid, and is_admitted is exactly the predicate “is the bottom element.” The submonoid $\langle L \rangle$ generated by the seven named nonzero constants together with ADMITTED is isomorphic to $\mathbb{D}_7$ of Definition ?? via “which named lanes are nonzero.”*

Proof. Bitwise OR on `u64` is associative, commutative, idempotent, with identity the zero word; compose inherits these, so $\mathbb{D}$ is a bounded commutative idempotent monoid whose bottom is ADMITTED , and $\text{is_admitted}(d) \iff d.0 = 0 \iff d = \text{ADMITTED}$. Each named constant c_i ($1 \leq i \leq 7$) has $c_i.0 = 0\text{xFF} \ll 8i$ occupying lane i alone, so for any subset $S \subseteq \{1, \dots, 7\}$ the composite $\bigvee_{i \in S} c_i$ has byte lane i nonzero iff $i \in S$. The map $\bigvee_{i \in S} c_i \mapsto \mathbf{1}_S \in \mathbb{D}_7$ (the indicator of S) is therefore a well-defined bijection $\langle L \rangle \rightarrow \mathbb{D}_7$ carrying compose to $\vee$ and ADMITTED to $\mathbf{0}$: a monoid isomorphism. $\square$

Remark. Definition ?? uses eight byte-lanes but only seven carry a named denial; lane 0 is the region of the clean word and is never set by a denial constant. Thus the *word* has seven informative lanes, while the *category* taxonomy of Chapter ?? has eight buckets, the eighth (`Reserved`) being an uninhabited slack coordinate. The two “eights” are the byte width, not a claim of eight active denials; we are careful never to conflate them.

2.3 The fired-mask projection

The keystone’s agent byte and its planetary admission sweep require collapsing a wide, possibly multi-lane word onto a fixed-width bit vector without losing *which* lanes fired. This is `to_fired_mask`.

Definition 2.3 (Fired-mask). $\pi_f : \mathbb{D} \rightarrow \{0, 1\}^8$ sends a word d to the bit vector whose bit j equals the nonzero-indicator of byte lane j of d : $\pi_f(d)_j = \mathbf{1}[(d.0 \gg 8j) \& 0\text{xFF} \neq 0]$. The *praxis* realization computes this branchlessly via the identity $\mathbf{1}[x \neq 0] = (x \mid (-x)) \gg 63$ on each lane.

Theorem 2.3 (Fired-mask is a semilattice homomorphism). π_f is a homomorphism of bounded commutative idempotent monoids: $\pi_f(\text{ADMITTED}) = \mathbf{0}$ and, for all $a, b \in \mathbb{D}$,

$$\pi_f(\text{compose}(a, b)) = \pi_f(a) \vee \pi_f(b).$$

Restricted to $\langle L \rangle$ it is a monoid isomorphism onto the seven-bit image $\{0\} \times \{0, 1\}^7 \subseteq \{0, 1\}^8$.

Proof. $\pi_f(\text{ADMITTED}) = \mathbf{0}$ since every lane of the zero word is zero. Byte lane j of $\text{compose}(a, b) = a.0 \mid b.0$ is $a_j \mid b_j$ (bitwise OR acts lane-locally), and for byte values $\mathbf{1}[a_j \mid b_j \neq 0] = \mathbf{1}[a_j \neq 0] \vee \mathbf{1}[b_j \neq 0]$ because a disjunction of bytes is nonzero iff some operand is; hence $\pi_f(a \mid b)_j = \pi_f(a)_j \vee \pi_f(b)_j$ for every j , which is the claimed homomorphism law. On $\langle L \rangle$, Proposition ?? identifies elements with subsets $S \subseteq \{1, \dots, 7\}$, and π_f sends such an element to the indicator of S in bits $1 \dots 7$ (bit 0 always zero); this is a bijection onto $\{0\} \times \{0, 1\}^7$ preserving join and bottom. $\square$

Theorem ?? is the projection principle at the algebra layer: an arbitrarily composed denial word — the aggregate of an entire pipeline (Chapter ??) — projects losslessly, as far as *which lanes fired*, onto at most eight bits. This is the homomorphism that makes the keystone’s word-parallel fleet-admission sweep sound (Proposition ??).

Chapter 3

Obligations as Lane Maps

3.1 Obligations and total denial

Definition 3.1 (Obligation and lane map). An *obligation* is a first-class, hashable value the payload must satisfy before admission. In `praxis` the `Obligation` enum has exactly three kinds: `Precondition{predicate_id, params_hash}`, `BlockingConstraint{reason}`, and `EvidenceRequired{evidence_type}`. Each obligation g induces a *lane map* $\delta_g : \mathcal{O} \rightarrow \mathbb{D}$ with

$$\delta_g(o) = \begin{cases} \text{ADMITTED} & \text{if } g \text{ is satisfied by } o, \\ \ell(g) & \text{otherwise,} \end{cases}$$

where $\ell(g) \in \mathbb{D}$ is the lane g fires on failure. For an obligation *set* $G = \{g_1, \dots, g_m\}$ the *total denial* is the join

$$d_G(o) = \text{compose_denials}(\{\delta_{g_i}(o) : g_i \text{ unmet}\}) = \bigvee_{i=1}^m \delta_{g_i}(o).$$

The right-hand fold is literally `compose_denials` in `refusal.rs`: it maps each refusal scenario to its lane via `denial_lane` and folds with `compose` from the seed `ADMITTED`. Because `ADMITTED` is the monoid identity (Proposition ??), an empty set of unmet obligations composes to `ADMITTED` — the admitted word — which is exactly the documented behavior (“empty input composes to `ADMITTED`”).

Proposition 3.1 (Bottom characterizes admission). $\alpha_G(o) \neq \perp \iff d_G(o) = \text{ADMITTED} \iff$ every $g_i \in G$ is satisfied by o .

Proof. $d_G(o) = \bigvee_i \delta_{g_i}(o) = \text{ADMITTED}$ iff every summand is `ADMITTED` (a join of elements of a join-semilattice equals the bottom iff each element is the bottom, by Proposition ??), i.e. iff no obligation is unmet. Admission returns non-refusal exactly in that case (Axiom ??). $\square$

3.2 Monotonicity of admission

Theorem 3.2 (Admission is antitone in the obligation set). Let $G \subseteq G'$ be obligation sets. Then for every observation o ,

$$d_G(o) \preceq d_{G'}(o), \quad \text{hence} \quad \mathcal{O}_{G'}^* \subseteq \mathcal{O}_G^*,$$

where $\mathcal{O}_G^* = \{o : d_G(o) = \text{ADMITTED}\}$ is the admitted set under G . Adding obligations can only enlarge denial and shrink admission; it can never admit an observation a smaller obligation set refused.

Proof. $d_{G'}(o) = \bigvee_{g \in G'} \delta_g(o) = \left(\bigvee_{g \in G} \delta_g(o) \right) \vee \left(\bigvee_{g \in G' \setminus G} \delta_g(o) \right) = d_G(o) \vee d_{G' \setminus G}(o) \succeq d_G(o)$, since in a join-semilattice $x \preceq x \vee y$ always (Proposition ??). If $o \in \mathcal{O}_{G'}^*$ then $d_{G'}(o) = \text{ADMITTED}$, the bottom; from $d_G(o) \preceq d_{G'}(o) = \text{ADMITTED}$ and minimality of ADMITTED we get $d_G(o) = \text{ADMITTED}$, i.e. $o \in \mathcal{O}_G^*$. $\square$

Corollary 3.3 (Admission is an antitone map of lattices). *The assignment $G \mapsto \mathcal{O}_G^*$ is an order-reversing map from the lattice of obligation sets $(2^{\mathcal{G}}, \subseteq)$ to the lattice of admitted subsets $(2^{\mathcal{O}}, \subseteq)$: $G \subseteq G' \Rightarrow \mathcal{O}_{G'}^* \subseteq \mathcal{O}_G^*$. In particular the empty obligation set admits everything ($\mathcal{O}_{\emptyset}^* = \mathcal{O}$), and enlarging obligations monotonically tightens the gate.*

Proof. Order-reversal is Theorem ?? . With $G = \emptyset$ the join is empty, so $d_{\emptyset}(o) = \text{ADMITTED}$ for all o and $\mathcal{O}_{\emptyset}^* = \mathcal{O}$. $\square$

3.3 The obligation-to-scenario map is total

Proposition 3.4 (Obligation classification is compiler-certified total). *The map $\text{scn} : \text{Obligation} \rightarrow \mathcal{S}$ implemented by `From<Obligation> for RefusalScenario` is a total function realized by a wildcard-free `match` over the three *Obligation* kinds:*

Precondition $\mapsto$ *UnsatisfiedPrecondition*, *BlockingConstraint* $\mapsto$ *BlockingConstraint*, *EvidenceRe*

Because the match has no wildcard arm, adding a fourth Obligation kind without extending scn is a compile error; totality is therefore a static property of the program, not a runtime hope.

Proof. A Rust `match` on a closed enum type-checks only if every constructor is covered or a wildcard is present; the cited `match` covers all three constructors and uses no wildcard, so the compiler certifies exhaustiveness (this is the mechanized-totality mechanism formalized in Paper 00). Each arm returns a specific `RefusalScenario`, so `scn` is a total function. $\square$

Proposition ?? is the first half of the totality argument: every obligation *failure* becomes a scenario. The second half — every scenario becomes a category — is Chapter ??.

Chapter 4

The Eight-Bucket Taxonomy as a Surjection

4.1 Categories and scenarios

Definition 4.1 (The taxonomy). The *category set* is the eight-element enum

$\mathcal{C} = \{\text{Identity}, \text{Capacity}, \text{Topology}, \text{Temporal}, \text{Lifecycle}, \text{Authorization}, \text{Prerequisites}, \text{Reserved}\}.$

The *scenario set* $\mathcal{S}$ is the thirteen-variant `RefusalScenario` enum, partitioned by origin into: three *obligation-driven* variants (`BlockingConstraint`, `MissingEvidence`, `UnsatisfiedPrecondition`); seven *denial-lane* variants, one per named nonzero lane of Definition ?? (`WatchdogDrained`, `PreconditionFailed`, `SlaBreach`, `AuthorizationDenied`, `ResourceExhausted`, `ObjectLifecycleViolation`, `ConformanceGateFailed`); and three *logic/andon* variants (`KernelDenied`, `KernelInvalid`, `AndonInvariantViolated`). The *category map* $\text{cat} : \mathcal{S} \rightarrow \mathcal{C}$ is `RefusalScenario::category`.

Scenario	cat($\cdot$)	origin
<code>KernelInvalid</code>	<code>Identity</code>	logic
<code>ResourceExhausted</code>	<code>Capacity</code>	denial lane
<code>ConformanceGateFailed</code>	<code>Topology</code>	denial lane
<code>AndonInvariantViolated</code>	<code>Topology</code>	andon
<code>WatchdogDrained</code> , <code>SlaBreach</code>	<code>Temporal</code>	denial lanes
<code>BlockingConstraint</code> , <code>ObjectLifecycleViolation</code>	<code>Lifecycle</code>	obligation / lane
<code>AuthorizationDenied</code> , <code>KernelDenied</code>	<code>Authorization</code>	lane / logic
<code>MissingEvidence</code> , <code>UnsatisfiedPrecondition</code> , <code>PreconditionFailed</code>	<code>Prerequisites</code>	obligation / lane
<i>(none)</i>	<code>Reserved</code>	—

4.2 The totality theorem

Theorem 4.1 (Mechanized totality of classification). *The category map $\text{cat} : \mathcal{S} \rightarrow \mathcal{C}$ satisfies:*

- (i) **Totality.** *cat is a total function: every one of the thirteen scenarios has exactly one category.*

(ii) **Mechanization.** *cat* is realized by a wildcard-free *match*, so its totality is compiler-certified: adding a scenario variant without extending *cat* fails to compile. (The *praxis* test suite additionally pins each variant’s value.)

(iii) **Image.** $\text{im cat} = C \setminus \{\text{Reserved}\}$: exactly the seven buckets *Identity*, *Capacity*, *Topology*, *Temporal*, *Lif* are inhabited, and *Reserved* has empty preimage.

Consequently *cat* is surjective onto the inhabited sub-taxonomy but is not surjective onto *C*; the deficiency *Reserved* is exactly one bucket.

Proof. (i)–(ii) A Rust *match* on the closed *RefusalScenario* enum type-checks only under exhaustiveness; the *category* method matches all thirteen constructors with no wildcard arm, so the compiler certifies that every scenario is assigned, and each arm returns a single *RefusalCategory*, making *cat* total and single-valued. (iii) Reading the thirteen arms (Definition ?? table) exhibits a preimage for each of the seven listed buckets — e.g. $\text{KernelInvalid} \in \text{cat}^{-1}(\text{Identity})$, $\text{ResourceExhausted} \in \text{cat}^{-1}(\text{Capacity})$, and so on down the table — so all seven are in the image; and no arm returns *Reserved*, so $\text{cat}^{-1}(\text{Reserved}) = \emptyset$. Hence im cat is precisely the seven-element complement of *Reserved*. $\square$

Remark (Honesty of the reserved bucket). Theorem ??(iii) is deliberately *not* a claim of 8/8 surjectivity. The codomain names a bucket the mechanism does not yet inhabit, and the mathematics records that faithfully rather than inflating coverage. This is the doctrine’s antibody clause at the type level: a reserved coordinate is declared, unused, and provably unused, so no reader can be misled into thinking eight kinds of refusal are in force when seven are. When a future enforcement lane populates *Reserved*, the theorem’s image statement is what must be re-verified.

4.3 Where the type guarantee ends: the partial lane decoder

Not every total map in this taxonomy is a wildcard-free *match*, and the exception is instructive: it marks the precise boundary between what the type system certifies and what a runtime test must certify.

Definition 4.2 (Single-lane set and the lane decoder). Let $L = \{\text{ADMITTED}, c_1, \dots, c_7\} \subseteq \mathbb{D}$ be the clean word together with the seven named single-lane constants. The *lane decoder* $\text{scn}_\ell : \mathbb{D} \rightarrow \mathcal{S}$ is *scenario_for_denial_lane*: it returns *None* on *ADMITTED*, returns the matching denial-lane scenario on each c_i , and returns *None* on every word not in L .

Proposition 4.2 (The lane decoder cannot be a total *match*, and why). *scn_ℓ* is a *partial map*, definite (as *Some*) only on $L \setminus \{\text{ADMITTED}\}$. It is not realizable as a wildcard-free exhaustive *match*, because its domain $\mathbb{D}$ is a *#[repr(transparent)]* newtype over *u64* — an open value space of cardinality 2^{64} , not a closed enum — so the compiler cannot enumerate its cases. In particular any composed multi-lane word $d = c_i \vee c_j \notin L$ (a legitimate element produced by *compose*) has $\text{scn}_\ell(d) = \text{None}$: the decoder recognizes single lanes only. Exhaustiveness over the seven single lanes is therefore a runtime test obligation (discharged by the *category_mapping_is_total_over_denial_lanes* test), not a static type obligation.

Proof. Rust’s exhaustiveness checker operates on closed sum types; a newtype over `u64` exposes 2^{64} inhabitants with no finite constructor set, so no wildcard-free `match` on its value is well-typed — the implementation is necessarily an `if/else` chain over the eight known constants with a `None` fallback. On `ADMITTED` it returns `None` by the first guard; on each c_i it returns the corresponding `Some` scenario; on any other word (including every $c_i \vee c_j$ with $i \neq j$, which lies outside L) it falls through to `None`. Thus $\text{dom}^+ \text{scn}_\ell = L \setminus \{\text{ADMITTED}\}$ where dom^+ denotes the `Some`-domain. $\square$

Proposition 4.3 (The lane encoder is total and a section). *The lane encoder $\text{lane} : \mathcal{S} \rightarrow \mathbb{D}$ implemented by `denial_lane` is a total function realized by a wildcard-free `match` over all thirteen scenarios. Restricted to the seven denial-lane scenarios $\mathcal{S}_\ell$, it and scn_ℓ form a section–retraction pair:*

$$\text{scn}_\ell \circ \text{lane} = \text{id}_{\mathcal{S}_\ell} \quad \text{and} \quad \text{lane} \circ \text{scn}_\ell = \text{id}_{L \setminus \{\text{ADMITTED}\}},$$

so the seven single lanes and the seven denial-lane scenarios are in bijection. The remaining six scenarios (obligation-driven and logic/andon) are mapped by lane onto closest-fit lanes and are therefore not in the range of that bijection.

Proof. lane matches all thirteen constructors with no wildcard, hence is total (same mechanism as Theorem ??). For each of the seven denial-lane scenarios s , $\text{lane}(s) = c_i$ is its own lane, and $\text{scn}_\ell(c_i) = s$ by Definition ??, giving $\text{scn}_\ell(\text{lane}(s)) = s$; conversely $\text{lane}(\text{scn}_\ell(c_i)) = \text{lane}(s) = c_i$. Both round-trips are the identity on their respective seven-element sets, the round-trip pinned by the cited test. The six non-lane scenarios map by lane onto lanes already claimed by the seven (e.g. `MissingEvidence` $\mapsto$ `PRECONDITION_FAILED`), so lane is not injective overall and those six are outside the bijection. $\square$

Corollary 4.4 (The assembled totality statement). *Composing the total maps of Propositions ?? and ?? with the total classification of Theorem ??: every unmet obligation (via $\text{cat} \circ \text{scn}$) and every non-`ADMITTED` single denial lane (via $\text{cat} \circ \text{scn}_\ell$ on $L \setminus \{\text{ADMITTED}\}$) maps to exactly one of the seven inhabited categories. No obligation failure and no fired single lane is left unclassified.*

Proof. Immediate by composition of the three total (respectively `Some`-total) maps, each single-valued into its codomain, landing in $\text{im cat} = \mathcal{C} \setminus \{\text{Reserved}\}$ (Theorem ??). $\square$

Corollary ?? is precisely the totality claim demanded of a fleet vocabulary (D3): a refusal never falls through a crack, and its class is legible to any recipient who holds the eight-element enum, regardless of the sender’s interior.

Chapter 5

Composition Across a Pipeline as a Monoid Action

5.1 Pipelines and their aggregate denial

A manufacture rarely faces a single gate. It traverses a *pipeline*: **Judge** evaluates obligations; **Admit** may deny; a **prolog8** kernel gate may refuse a logical side-condition (Chapter ??); an optional andon second-gate ring may fire. Each stage emits a denial word; the fleet needs one.

Definition 5.1 (Pipeline and stage denial). Let **Stage** be the set of admission stages. Fix an observation o and let $\varphi_o : \mathbf{Stage} \rightarrow \mathbb{D}$ send a stage to the denial word it emits on o (via its scenarios and `denial_lane`). A *pipeline* is a finite sequence $w = s_1 s_2 \cdots s_k \in \mathbf{Stage}^*$, an element of the free monoid on **Stage** under concatenation with unit the empty sequence ε . Its *aggregate denial* is

$$\Phi_o(w) = \text{compose_denials}(\varphi_o(s_1), \dots, \varphi_o(s_k)) = \bigvee_{i=1}^k \varphi_o(s_i), \quad \Phi_o(\varepsilon) = \text{ADMITTED}.$$

Theorem 5.1 (Pipeline denial is the free-monoid homomorphism). $\Phi_o : (\mathbf{Stage}^*, \cdot, \varepsilon) \rightarrow (\mathbb{D}, \text{compose}, \text{ADMITTED})$ is the unique monoid homomorphism extending φ_o along the inclusion $\mathbf{Stage} \hookrightarrow \mathbf{Stage}^*$. Because the target is commutative and idempotent, Φ_o factors through the finite join-semilattice generated by $\{\varphi_o(s) : s \in \mathbf{Stage}\}$; hence $\Phi_o(w)$ depends only on the set of distinct stage-denials occurring in w — it is invariant under reordering the stages and under repeating them.

Proof. $\Phi_o(\varepsilon) = \text{ADMITTED}$ and $\Phi_o(w \cdot w') = \bigvee_{s \in w} \varphi_o(s) \vee \bigvee_{s \in w'} \varphi_o(s) = \Phi_o(w) \vee \Phi_o(w') = \text{compose}(\Phi_o(w), \Phi_o(w'))$ by associativity of $\vee$; so Φ_o is a monoid homomorphism, and it agrees with φ_o on length-one words. Uniqueness is the universal property of the free monoid: any homomorphism agreeing with φ_o on generators is determined on all of $\mathbf{Stage}^*$ by the homomorphism laws. For the factorization, commutativity gives order-independence ($a \vee b = b \vee a$) and idempotence gives multiplicity-independence ($a \vee a = a$), so $\Phi_o(w) = \bigvee_{s \in \text{set}(w)} \varphi_o(s)$ depends only on the underlying set of distinct denials. $\square$

Corollary 5.2 (A pipeline admits iff every stage admits). $\Phi_o(w) = \text{ADMITTED} \iff \varphi_o(s_i) = \text{ADMITTED}$ for every stage s_i in w . A single refusing stage refuses the whole pipeline, and the aggregate word records every lane that fired anywhere along w .

Proof. A join in a join-semilattice equals the bottom iff every joinand is the bottom (Proposition ??); apply to $\Phi_o(w) = \bigvee_i \varphi_o(s_i)$. The support of the aggregate is $\text{supp } \Phi_o(w) = \bigcup_i \text{supp } \varphi_o(s_i)$, the union of the per-stage fired lanes. $\square$

Corollary ?? is the correctness statement for `ReceiptMeta.denial`: when a manufacture is receipted, the honest denial word written into the `OcelCausalFrame` is the aggregate $\Phi_o(w)$, so the receipt records exactly what was waved through — not a fiction of clean admission when a non-fatal lane fired. Composition (D1) is thereby a theorem, not a convention.

5.2 The fleet sweep is word-parallel

Proposition 5.3 (Byte-packed fleet admission). *Let a fleet of N manufactures produce aggregate words $\Phi^{(1)}, \dots, \Phi^{(N)}$, and pack each into a byte $b^{(r)} = \pi_f(\Phi^{(r)}) \in \{0, 1\}^8$. Then:*

- (i) $\pi_f(\Phi_o(w)) = \bigvee_i \pi_f(\varphi_o(s_i))$: *the fleet byte of a pipeline equals the bitwise OR of its stage bytes (fired-mask commutes with aggregation).*
- (ii) *A fleet admission sweep — “are all agents clean?” — is one linear pass computing $\bigvee_r b^{(r)}$, eight agents per 64-bit word, and returns **0** iff every agent is admitted.*

Proof. (i) π_f is a homomorphism (Theorem ??) and Φ_o is a homomorphism (Theorem ??); their composite carries `compose` to $\vee$, so $\pi_f(\bigvee_i \varphi_o(s_i)) = \bigvee_i \pi_f(\varphi_o(s_i))$. (ii) The pass folds $\vee$ over N bytes; by Corollary ?? applied fleet-wide, the fold is **0** iff each $b^{(r)} = \mathbf{0}$ iff each aggregate is `ADMITTED` iff every agent is admitted. Packing eight bytes per machine word makes the fold word-parallel. $\square$

Proposition ?? is the algebraic underpinning of the keystone’s “agent as eight bits” construction and its memory-bandwidth-bounded planetary sweep: the reason a fleet of 10^{12} agents can be admission-swept by masked ORs is that both aggregation (over stages) and projection (to bits) are semilattice homomorphisms, so the cheap bit-level operation computes the same verdict as the expensive per-stage evaluation.

Chapter 6

Proof-Carrying Admission via SLD-Resolution

6.1 The logical quarantine

Some admissions are not Boolean batteries but *logical queries*: “does this atom follow from the admitted facts and rules?” `praxis` routes these to the `prolog8` kernel (`run_kernel_query` in `ops.rs`). Admission here is derivation, and — unlike a bare Boolean gate — it can carry a *proof*. First the language must be quarantined.

Definition 6.1 (Logic admission; the Rice quarantine for Horn logic). `prolog8` admits a query, fact block, or rule only if it lies in a bounded, decidable Horn fragment: arity ≤ 8 , rule body ≤ 8 atoms, ≤ 8 variables, proof fan-in ≤ 8 ; no cut, no dynamic mutation (`assert/retract`), stratified negation, bounded or declared recursion, no runtime text parsing, no side effects, interned (non-string) terms. Violations are enumerated by `RejectionCode` (a closed enum of ~ 30 structured codes: `ArityCapExceeded`, `RuleBodyCapExceeded`, `CutNotAdmitted`, `UnstratifiedNegation`, `DynamicMutationNotAdmitted`, ...). `admit_atom` and `admit_rule` run before any query touches the kernel.

Definition ?? is Rice’s theorem [?] answered at the logic layer: full first-order or unrestricted Prolog derivability is undecidable, so the kernel retracts onto a fragment whose byte caps make the search space finite, and everything outside the fragment is refused with a code. This is the same move as $\alpha : \mathcal{O} \rightarrow \mathcal{O}^* \cup \{\perp\}$ (Paper 00), specialized to logic and carrying, additionally, a proof on the inside.

6.2 The proof-carrying trichotomy

Definition 6.2 (Query result). `Kernel::query` returns `QueryResult`, one of: `Answered(Vec<Decision>)`, `Denied(Box<Decision>)`, or `Invalid(RejectionCode)`. A `Decision` carries a *proof* (a `Vec<ProofNode>` DAG, positive or negative) and a *receipt* sufficient for deterministic replay.

Theorem 6.1 (Proof-carrying admission trichotomy). *For a query q evaluated against an admitted kernel (admitted catalog, fact blocks, and rules), exactly one of the following holds:*

- (a) **Answered.** q passes `admit_atom` and unifies with a stored fact or is derivable by a depth-one rule application; then $\text{query}(q) = \text{Answered}(D_1, \dots, D_p)$ with $p \geq 1$, each D_j carrying a positive proof DAG whose every node has $\text{fan-in} \leq 8$, plus a receipt.
- (b) **Denied.** q passes `admit_atom` but no fact and no admitted rule derive it; then $\text{query}(q) = \text{Denied}(D)$, where D carries a negative proof — a finite, closed witness that the depth-bounded SLD search was exhausted without success — plus a receipt.
- (c) **Invalid.** q fails `admit_atom`; then $\text{query}(q) = \text{Invalid}(c)$ with $c \in \text{RejectionCode}$, and no proof is produced.

Thus a **Denied** query yields a proof tree; an **Invalid** query yields a rejection code.

Proof. Trace `Kernel::query`. Its first action is the admission gate: if `admit_atom(q)` returns `Err(c)` the kernel returns `Invalid(c)` immediately, before any search, establishing (c) and its no-proof clause. Otherwise the query is in the admitted fragment and the kernel runs three phases. *Phase 1* scans fact blocks for rows unifying with q under its bound positions; a nonempty match set is assembled into **Answered** decisions, each with a positive proof rooted at the matched fact hash. *Phase 2*, on empty *Phase 1*, applies each rule whose head unifies with q by depth-one backward chaining with backtracking over body atoms (a single SLD-resolution step against the fact base); any successful derivation is assembled into **Answered**, giving (a). The fan-in bound holds because `admit_rule` rejects any rule or proof node exceeding fan-in 8 (`ProofFanInExceeded`), so every emitted proof node has ≤ 8 children. *Phase 3* is reached iff both searches are empty; the kernel then assembles a **Denied** decision with a *negative* proof recording the exhausted search, giving (b). The three phases are mutually exclusive by construction (each returns), so exactly one branch fires. Soundness and completeness of the depth-bounded resolution — that *Phase 3* is reached *only* when q is underivable within the fragment — is SLD soundness and completeness for definite/stratified Horn programs [?, ?], restricted to the finite search space that the byte caps of Definition ?? guarantee. $\square$

Proposition 6.2 (Boundedness gives a bounded certificate). *Every proof node emitted by the kernel has fan-in ≤ 8 and every rule body has ≤ 8 atoms, so a proof (positive or negative) is a bounded-degree DAG; its rolling `proof_root` is a single fixed-width hash field of the receipt. A bounded-context verifier checks a query outcome by recomputing `proof_root` and comparing — cost $O(\kappa)$, independent of the size of the fact base — and the deterministic replay path recomputes exactly this root.*

Proof. Fan-in and body caps are enforced by `admit_rule` (`ProofFanInExceeded`, `RuleBodyCapExceeded`), so proof nodes have bounded out-degree; the receipt commits the proof by its root hash (a projection of the DAG onto one field), and replay recomputes the root from the stored decision, agreeing iff untampered — the Faithful Projection mechanism of the keystone applied to the proof DAG. The verifier reads the fixed-width root, not the DAG, so its cost is $O(\kappa)$. $\square$

Proposition ?? is the projection principle at the logic layer: an arbitrarily large search over facts and rules projects onto one root hash a bounded verifier can check, so proof-carrying admission scales the same way byte-level admission does.

6.3 Logic refusals rejoin the monoid

The logical layer does not live apart from Chapters ??–??. its refusals are ordinary denial words.

Proposition 6.3 (Kernel outcomes embed into the denial monoid). *The refusing branches of Theorem ?? embed into $(\mathbb{D}, \text{compose}, \text{ADMITTED})$ via the taxonomy: `run_kernel_query` maps a *Denied* outcome to the scenario *KernelDenied* (category *Authorization*) and an *Invalid* outcome to *KernelInvalid* (category *Identity*); by `denial_lane` both compose into the lane *CONFORMANCE_GATE_FAILED*. Hence a logical refusal is an element of $\mathbb{D}$ subject to monotonicity (Theorem ??), classification (Theorem ??), pipeline composition (Theorem ??), and fired-mask projection (Theorem ??), exactly as any obligation refusal.*

Proof. `run_kernel_query` constructs `RefusalScenario::KernelDenied` on the *Denied* branch and `RefusalScenario::KernelInvalid` on the *Invalid* branch (Definition ??); their categories are as stated by Theorem ??, and `denial_lane` sends both to *CONFORMANCE_GATE_FAILED* (Proposition ??, closest-fit clause). Being elements of $\mathbb{D}$, they are acted on by all the structure of Chapters ??–??. $\square$

Remark (The epistemic distinction, made precise). The trichotomy separates three epistemic states. *Invalid* means *outside the language*: a reason (a `RejectionCode`) but no proof — the query was never well-formed enough to have a truth value in the fragment. *Denied* means *in the language and false*: a negative proof witnesses underivability. *Answered* means *in the language and true*: a positive proof witnesses derivability. Only membership-in-the-language is a type-like gate (`admit_atom`); truth or falsity within the language is decided by bounded search and *always* ships a witness. This is the sharpest form of “refusal is data”: at the logic layer even a denial carries its proof, so a bounded verifier can trust the “no” without re-running the search.

Chapter 7

The Boundary Observation Algebra and the Domain Proposer

7.1 Untrusted observations and the boundary

A subtle consequence of the admission algebra is that observations arriving from outside a system boundary carry no authority by default. The `praxis-proposer` crate formalizes this by distinguishing two regimes.

Definition 7.1 (Observation authority). Let $\mathcal{O}$ be the raw observation space (Axiom ??). An observation $o \in \mathcal{O}$ is *authoritative* ($o \in \mathcal{O}^*$) if it was produced by an admitted actuation with a chained receipt — equivalently, if it carries a committed payload digest inside an `OcelCausalFrame`. All other observations are *untrusted* ($o \in \mathcal{O} \setminus \mathcal{O}^*$). The proposer’s admission map α_{prop} retracts $\mathcal{O}$ onto $\mathcal{O}_{\text{prop}}^*$ by treating every inbound observation as untrusted until its receipt chain satisfies the obligation battery G_{prop} .

Proposition 7.1 (Proposer boundary separation). *Under Definition ??, the `praxis-proposer` enforces the invariant $\text{im } \mu_{\text{prop}} \subseteq \mathcal{O}_{\text{prop}}^*$: no proposal is manufactured from an unadmitted observation. Operationally, the generic `Domain<D>` proposer evaluates its *Obligation* battery on every inbound record before calling `Admit::admit`; a record that fails any obligation is refused with its denial word and never reaches the manufacturing step.*

Proof. The `Domain` struct holds a `Law<Obligation>` attached to its typed domain `D`. Proposing calls `judge` then `admit` in sequence (the lifecycle arrows j, a of the foundations paper). An `Andon::Halted` on either step returns the `Err` branch with its denial word; the manufacturing step (`D::manufacture`) is unreachable from the `Err` branch. Hence $\text{im } \mu_{\text{prop}}$ is contained in `Admitted` objects, elements of $\mathcal{O}_{\text{prop}}^*$. $\square$

7.2 Maximum Reachable Revenue: additive separation over accounts

Definition 7.2 (Maximum Reachable Revenue). Let A be a set of client accounts. For each $a \in A$, let `lawful_targets(a)` be the evidence-gated target stages for account a , and let

$\text{realized}(a, s)$ be the revenue function under target stage s . The *Maximum Reachable Revenue* (MRR) of the fleet is

$$\text{MRR} = \max_{\text{plan}} \sum_{a \in A} \text{realized}(a) .$$

Theorem 7.2 (MRR additively separates over independent accounts). *If the accounts in A are independent — no account’s target stages share resources or evidence with another’s — then*

$$\text{MRR} = \sum_{a \in A} \max_{s \in \text{lawful_targets}(a)} \text{realized}(a, s) ,$$

reducing the joint plan search from exponential ($\prod_a |\text{lawful_targets}(a)|$) to linear ($\sum_a |\text{lawful_targets}(a)|$).

Proof. Independence means the feasibility of account a ’s targets is unaffected by the targets of any $b \neq a$. Hence the joint optimization $\max_{\text{joint}} \sum_a r(a)$ decomposes into $\sum_a \max_{s_a} r(a, s_a)$ by linearity of the sum and independence of the maxima. In **praxis-proposer** this is realized in `mrr.rs: compute_mrr` iterates over accounts, calls `max_realized_revenue(a)` for each, and sums the per-account maxima. No cross-account joint optimization is performed. $\square$

Corollary 7.3 (Boundary independence is the MRR precondition). *Theorem ?? holds exactly when no obligation in G_{prop} couples distinct accounts (e.g. a shared authority token or a mutually exclusive resource). If such coupling exists, the proposer’s obligation battery must be extended with a **BlockingConstraint** on the shared resource, which by Theorem ?? only tightens the admitted set and preserves the denial algebra.*

Proof. Immediate from Theorem ??: adding an obligation enlarges denial and shrinks admission. With the extended battery the coupled accounts are refused by the coupling constraint and the remaining independent ones decompose additively as before. $\square$

Remark (The generic `Domain<D>` proposer). The `Domain` struct in **praxis-proposer** is a generic adapter parameterized by `DomainSpec`, a sealed trait that supplies the domain’s typed observation schema and `manufacture`. Any domain (revenue, scheduling, compliance) inherits the full denial algebra of Chapters ??–?? for free: its refusals compose under `DenialPolarity`, are classified by `RefusalCategory`, and are swept byte-parallel across a fleet. The MRR theorem (Theorem ??) is the domain-neutral consequence of this inheritance: account-level independence is a structural claim about the obligation battery, independent of what the domain manufactures.

Chapter 8

Conclusion

We set out to give the refusal object the algebra its role in a trillion-agent fleet demands, and we have discharged each of the four demands of Chapter ?? as a theorem anchored to **praxis**.

Composition (D1) is Theorem ??: pipeline denial is the unique monoid homomorphism from the free monoid on stages into the bounded commutative idempotent monoid $(\mathbb{D}, \text{compose}, \text{ADMITTED})$ (Propositions ??, ??), so aggregation is associative, order-independent, and multiplicity-independent, and a pipeline admits iff every stage admits (Corollary ??). *Classification (D2)* is the category map cat , and its *totality (D3)* is Theorem ??: a compiler-certified total classification onto exactly seven inhabited buckets plus a provably empty **Reserved** slack coordinate, assembled with the total obligation and lane maps into Corollary ?? — no refusal is unclassified. We located the exact boundary of the type system’s guarantee: the lane decoder is partial precisely because its domain is an open 2^{64} newtype, so its exhaustiveness is a test obligation, not a type obligation (Proposition ??). And *reason-carrying (D4)*, at its logical limit, is the proof-carrying trichotomy of Theorem ??: an admitted-but-false query ships a negative proof tree and an ill-formed query ships a rejection code, both bounded (Proposition ??) and both rejoining the denial monoid (Proposition ??).

Monotonicity (Theorem ??) ties the whole together: enlarging obligations only tightens the gate, so the algebra is safe to grow. The fired-mask homomorphism (Theorem ??, Proposition ??) is what lets the keystone’s planetary admission sweep compute the true verdict with a bitwise OR. What began, in the keystone, as one Construction and one Proposition is now a complete small theory: the space of “why not” is a join-semilattice, its classification is a mechanized surjection, its aggregation is a free-monoid homomorphism, and its logical instance carries proof.

A refusal is not the absence of an answer; it is an answer with an algebra.

Appendix A

Notation and Code Anchors

Symbols

Symbol	Meaning
$\mathbb{D}$, $\vee$, ADMITTED	denial-word monoid; join (<code>compose</code>); bottom (ADMITTED)
$\mathbb{D}_n$, $\text{supp } d$	abstract n -lane word space; fired-lane support
π_f	fired-mask projection $\mathbb{D} \rightarrow \{0, 1\}^8$ (<code>to_fired_mask</code>)
δ_g , $d_G(o)$	obligation lane map; total denial (join over G)
$\mathcal{O}_G^*$	admitted set under obligation set G
$\mathcal{C}$, $\mathcal{S}$	category set (8); scenario set (13)
cat , scn , lane , scn_ℓ	category map; obligation $\rightarrow$ scenario; scenario $\rightarrow$ lane; lane decoder
Φ_o , Stage *	pipeline aggregate denial; free monoid on stages
$\perp$, κ	refusal value; bounded-verifier context capacity

Mathematics to code

Object	Code artifact	Location
$(\mathbb{D}, \vee, \text{ADMITTED})$	<code>DenialPolarity</code> , <code>compose</code>	<code>bcinr-powl-receipt/src/denial.rs</code>
π_f	<code>DenialPolarity::to_fired_mask</code>	<code>bcinr-powl-receipt/src/denial.rs</code>
d_G , <code>compose_denials</code>	<code>compose_denials</code>	<code>crates/praxis-core/src/refusal.rs</code>
<code>cat</code> (Thm ??)	<code>RefusalScenario::category</code>	<code>crates/praxis-core/src/refusal.rs</code>
<code>scn</code> (Prop ??)	<code>From<&Obligation></code>	<code>crates/praxis-core/src/refusal.rs</code>
<code>scn$_\ell$</code> (Prop ??)	<code>scenario_for_denial_lane</code>	<code>crates/praxis-core/src/refusal.rs</code>
<code>lane</code> (Prop ??)	<code>denial_lane</code>	<code>crates/praxis-core/src/refusal.rs</code>
<code>Obligation</code> (3 kinds)	<code>Obligation</code> enum	<code>crates/praxis-core/src/law.rs</code>
<code>Andon::Halted</code>	<code>Andon</code> enum	<code>crates/praxis-core/src/law.rs</code>
<code>Trichotomy</code> (Thm ??)	<code>Kernel::query</code> , <code>QueryResult</code>	<code>prolog8/src/kernel.rs</code>
<code>Logic quarantine</code>	<code>admit_atom</code> , <code>RejectionCode</code>	<code>prolog8/src/admission.rs</code>
<code>Kernel refusals</code> (Prop ??)	<code>run_kernel_query</code>	<code>src/ops.rs</code>

Bibliography

- [1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.
- [2] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.
- [3] G. Birkhoff, *Lattice Theory*, 3rd ed., American Mathematical Society Colloquium Publications, vol. 25, 1967.
- [4] R. A. Kowalski, *Predicate logic as programming language*, Information Processing **74** (Proc. IFIP Congress), North-Holland, 1974, 569–574.
- [5] J. W. Lloyd, *Foundations of Logic Programming*, 2nd ed., Springer-Verlag, 1987.
- [6] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.
- [7] J. O’Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O’Hearn, *BLAKE3: One function, fast everywhere*, 2020.